

This extract is from Core Java Volume I, 13th ed. This is used for educational purposes only under the Fair Use copyright laws.

The body of the `main` method contains a statement that outputs a single line of text to the console.

Here, we are using the `System.out` object and calling its `println` method. Notice the periods used to invoke a method. Java uses the general syntax

object.method(arguments)

as its equivalent of a function call.

In this case, the `println` method receives a string argument. The method displays the string argument on the console. It then terminates the output line, so that each call to `println` displays its output on a new line. Notice that Java, like C/C++, uses double quotes to delimit strings. (You can find more information about strings later in this chapter.)

Methods in Java, like functions in any programming language, can use zero, one, or more *arguments*. Even if a method has no arguments, you must still use empty parentheses. For example, a variant of the `println` method with no arguments just prints a blank line. You invoke it with the call

```
System.out.println();
```



Note: `System.out` also has a `print` method that doesn't add a newline character to the output. For example, `System.out.print("Hello")` prints `Hello` without a newline. The next output appears immediately after the letter `o`.

3.2. Comments

Comments in Java, as in most programming languages, do not show up in the executable program. Thus, you can add as many comments as needed without fear of bloating the code. Java has three ways of marking

comments. The most common form is a `//`. Use this for a comment that runs from the `//` to the end of the line.

```
System.out.println("We will not use 'Hello, World!'); //  
is this too cute?
```

When longer comments are needed, you can mark each line with a `//`, or you can use the `/*` and `*/` comment delimiters that let you block off a longer comment.

Finally, a third kind of comment is used to generate documentation automatically. This comment uses a `/**` to start and a `*/` to end. You can see this type of comment in [Listing 3.1](#). For more on this type of comment and on automatic documentation generation, see [Chapter 4](#).

Listing 3.1 FirstSample/FirstSample.java

```
1  /**  
2   * This is the first sample program in Core Java Chapter 3  
3   * @version 1.01 1997-03-22  
4   * @author Gary Cornell  
5   */  
6  public class FirstSample  
7  {  
8      public static void main(String[] args)  
9      {  
10         System.out.println("We will not use 'Hello, World!');  
11     }  
12 }
```



Caution: `/* */` comments do not nest in Java. That is, you might not be able to deactivate code simply by surrounding it with `/*` and `*/` because the code you want to deactivate might itself contain a `*/` delimiter.



Tip: You can use the `JDK_JAVA_OPTIONS` environment variable to pass command-line options to the `java` launcher:

```
export JDK_JAVA_OPTIONS='--class-path /home/user/classdir -
enableassertions'
```



Caution: According to JEP 293, single-letter options without arguments can be grouped together:

```
jar -c -v -f myapp.jar -m MANIFEST.MF *.class
```

can be simplified to

```
jar -cv -f myapp.jar -m MANIFEST.MF *.class
```

But that does not currently work. Strangely,

```
jar -cvfm myapp.jar MANIFEST.MF *.class
```

works even though it should not. Until this is sorted out, it seems best to stick with the classic `tar`-style options for the `jar` command.



Caution: According to JEP 293, arguments of single-letter options can be separated by whitespace or directly follow the option:

```
javac -d outputdir . . .
```

or

```
javac -doutputdir . . .
```

The latter doesn't currently work. It also seems like a bad idea since it would invite conflicts with legacy options. For example, what should happen if the output directory happens to be named `eprecation`?

4.10. Documentation Comments

The JDK contains a very useful tool, called `javadoc`, that generates HTML documentation from your source files. In fact, the online API documentation described in [Chapter 3](#) is simply the result of running `javadoc` on the source code of the standard Java library.

If you add comments that start with the special delimiter `/**` to your source code, you too can easily produce professional-looking documentation. This is a very nice approach because it lets you keep your code and documentation in one place. If you put your documentation into a separate file, then, as

you probably know, the code and comments tend to diverge over time. When documentation comments are in the same file as the source code, it is an easy matter to update both and run `javadoc` again.

4.10.1. Comment Insertion

The `javadoc` utility extracts information for the following items:

- Modules
- Packages
- Public classes and interfaces
- Public and protected fields
- Public and protected constructors and methods

Protected members are introduced in [Chapter 5](#), interfaces in [Chapter 6](#), and modules in [Chapter 12](#).

You can (and should) supply a comment for each of these members. Each comment is placed immediately *above* the member it describes. A comment starts with a `/**` and ends with a `*/`.

Each `/** . . . */` documentation comment contains *free-form text* followed by *tags*. A tag starts with an `@`, such as `@since` or `@param`.

The *first sentence* of the free-form text should be a *summary statement*. The `javadoc` utility automatically generates summary pages that extract these sentences.

The most common `javadoc` tags are *block tags*. They must appear at the beginning of a line and start with `@`, optionally preceded by whitespace, the comment delimiter `/**`, or leading `*` which are often used for multi-line comments. In contrast, *inline tags* are enclosed in braces: `{@tagname contents}`. The contents may contain braces, but they must be balanced. Examples are the `@code` and `@link` tags.

4.10.2. Class Comments

The class comment must be placed *after* any `import` statements, directly before the class definition.

Here is an example of a class comment:

```
/**
 * A {@code Card} object represents a playing card, such
 * as "Queen of Hearts". A card has a suit (Diamond, Heart,
 * Spade or Club) and a value (1 = Ace, 2 . . . 10, 11 = Jack,
 * 12 = Queen, 13 = King).
 */
public class Card
{
    . . .
}
```



Note: There is no need to add an `*` in front of every line. For example, the following comment is equally valid:

```
/**
    A Card object represents a playing card, such
    as "Queen of Hearts". A card has a suit (Diamond, Heart,
    Spade or Club) and a value (1 = Ace, 2 . . . 10, 11 = Jack,
    12 = Queen, 13 = King).
*/
```

However, most IDEs supply the asterisks automatically and rearrange them when the line breaks change.

4.10.3. Method Comments

Each method comment must immediately precede the method that it describes. In addition to the general-purpose tags, you can use the following tags:

- `@param` *variable description*
This tag adds an entry to the “parameters” section of the current method. The description can span multiple lines and can use HTML tags. All `@param` tags for one method must be kept together.
- `@return` *description*
This tag adds a “returns” section to the current method. The description can span multiple lines and can use HTML tags.
- `@throws` *class description*
This tag adds a note that this method may throw an exception. Exceptions are the topic of [Chapter 7](#).

Here is an example of a method comment:

```
/**
 * Raises the salary of an employee.
 * @param byPercent the percentage by which to raise the salary (e.g., 10 means
 10%)
 * @return the amount of the raise
 */
public double raiseSalary(double byPercent)
{
    double raise = salary * byPercent / 100;
    salary += raise;
    return raise;
}
```



Tip: It can be tedious to write comments for methods whose description and return value are identical, such as:

```
/**
 * Returns the name of the employee.
 * @return the name of the employee
 */
```

In such cases, consider the inline form of `@return` introduced in Java 16:

```
/**
 * {@return the name of the employee}
 */
```

The description section becomes “Returns the name of the employee.”, and a “Returns” section with the same contents is added.

4.10.4. Field Comments

You only need to document public fields—generally that means static constants. For example:

```
/**
 * The "Hearts" card suit
 */
public static final int HEARTS = 1;
```

4.10.5. Package Comments

Place the class, method, and variable comments directly into the Java source files, delimited by `/**` . . . `*/` documentation comments. However, to generate *package* comments, you need to add a separate file in each package directory. You have two choices:

1. Supply a Java file named `package-info.java`. The file must contain an initial documentation comment, delimited with `/**` and `*/`, followed by a `package` statement. It should contain no further code or comments.
2. Supply an HTML file named `package.html`. All text between the tags `<body>` . . . `</body>` is extracted.

4.10.6. HTML Markup

In the free-form text, you can use HTML modifiers such as `<em>` . . . `</em>` for emphasis, `<strong>` . . . `</strong>` for strong emphasis, `<ul>`/`<li>` for bulleted lists, and `<img` . . . `</>` to include an image.

To type monospaced code, use `{@code . . . }` instead of `<code>. . . </code>`—then you don't have to worry about escaping `<` and `&` characters inside the code. If you want to write unescaped `<` or `&` in the plain font, use the `@literal{ . . . }` tag.

For multi-line code displays in an HTML `pre` tag, you can use:

```
/**
 * ...
 * <pre>{@code
 *     ...
 *     ...
 * }</pre>
 */
```

or, since Java 18:

```
/**
 * ...
 * {@snippet :
 *     ...
 *     ...
 * }
 */
```

Either way, you don't have to escape `<` or `&` characters in the code. However, braces have to match, and you cannot have `/* ... */` comments.

4.10.7. Links

You can use hyperlinks to other relevant parts of the `javadoc` documentation, or to external documents, with the `@see` and `@link` tags.

The tag `@see reference` adds a hyperlink in the “see also” section. It can be used with both classes and methods. Here, *reference* can be one of the following:

```
package.class#member label
<a href=". . .">label</a>
"text"
```

The first case is the most useful. You supply the name of a class, method, or variable, and `javadoc` inserts a hyperlink to the documentation. For example,

```
@see com.horstmann.corejava.Employee#raiseSalary(double)
```

makes a link to the `raiseSalary(double)` method in the `com.horstmann.corejava.Employee` class. You can omit the name of the package, or both the package and class names. Then, the member will be

located in the current package or class.

Note that you must use a #, not a period, to separate the class from the method or variable name. The Java compiler itself is highly skilled in determining the various meanings of the period character as separator between packages, subpackages, classes, inner classes, and methods and variables. But the javadoc utility isn't quite as clever, so you have to help it along.

Constructors have the special name <init>, not the name of the class, such as

```
@see com.horstmann.corejava.Employee#<init>()
```

You can specify an optional *label* after the member that will appear as the link anchor. If you omit the label, the user will see the member name.

If the @see tag is followed by a < character, then you need to specify a hyperlink. You can link to any URL you like. For example:

```
@see <a href="horstmann.com/corejava.html">The Core Java home page</a>
```

If the @see tag is followed by a " character, then the text is displayed in the “see also” section. For example:

```
@see "Core Java Volume 2"
```

You can add multiple @see tags for one member, but you must keep them all together.

If you like, you can place hyperlinks to other classes or methods anywhere in any of your documentation comments. Insert a tag of the form

```
{@link package.class#member}
```

anywhere in a comment. The member reference follows the same rules as for the @see tag.

Since Java 20, ids are automatically generated for level 2 and level 3 headings. For example,

```
<h2>General Principles</h2>
```

gets an id general-principles-heading, which you can refer from @see and @link tags. You need two # symbols to link to an id:

```
{@link com.horstmann.corejava.Employee##general-principles-heading}
```

Use @linkplain instead of @link if a link should be displayed in the plain font instead of the code font.



Note: If your comments contain links to other files, such as images (for example, diagrams or images of user interface components), place those files into a subdirectory, named doc-files, of the directory containing the source file. The javadoc utility will copy the doc-files

directories and their contents from the source directory to the documentation directory. You need to use the `doc-files` directory in your link, for example ``.

4.10.8. General Comments

The tag `@since text` makes a “since” entry. The *text* can be any description of the version that introduced this feature. For example, `@since 1.7.1`.

The following tags can be used in class documentation comments:

- `@author name`
This tag makes an “author” entry. You can have multiple `@author` tags, one for each author. Don't feel compelled to use this tag—your version control system does a more thorough job tracking authorship.
- `@version text`
This tag makes a “version” entry. The *text* can be any description of the current version.

The `{@value optionalFormat constantFieldReference}` inline tag inserts the value of a constant field, with an optional `printf`-style formatter since Java 20. For example, `{@value %X Integer#MAX_VALUE}` yields `7FFFFFFF`.

The `@deprecated` tag is used together with the `@Deprecated` annotation. It is followed by text that tells the user when and why the deprecation occurred, and to describe alternatives.

Finally, you can use the `{@index entry}` tag to add an entry to the generated index.

4.10.9. Code Snippets

The purpose of this feature is to allow you to import (presumably well-tested and up-to-date) code from external files. This minimizes the risk of incorrect or stale documentation. Code snippets were introduced in Java 18.

You can import an entire file with these tags:

```
{@snippet file=EmployeeTest.java}
{@snippet class=com.horstmann.corejava.EmployeeTest.java}
```

Files should be placed in the `snippet-files` subdirectory of the current package.

More commonly, you want to copy a *region* from a file. In the source file, you specify the name and extent of a region as follows:

```
public class EmployeeTest
{
    . . .
    // @start region=default-employee
```

```

    var e = new Employee();
    String name = e.getName(); // name is null
    // @end
    . . .
}

```

To highlight a part of the snippet, use a comment:

```

var e = new Employee(); // @highlight substring=new
String name = e.getName(); // @highlight regex=get[A-Z][a-z]+

```

Inside a region, you can replace a part with another string, for example to make the documentation more general:

```

var e = new Employee("Fred", 100000); // @replace regex=([^\s]) replacement="
(..., ...)"

```

Then the documentation contains:

```

var e = new Employee(..., ...);

```

To add a link, use:

```

var c = new Card(); // @link substring=Card target=com.horstmann.games#Card.
<init>()

```

These decorations and transformations are specified as comments, so that your source file compiles and runs.

4.10.10. Comment Extraction

Here, *docDirectory* is the name of the directory where you want the HTML files to go. Follow these steps:

1. Change to the directory that contains the source files you want to document. If you have nested packages to document, such as `com.horstmann.corejava`, you must be working in the directory that contains the subdirectory `com`. (This is the directory that contains the `overview.html` file, if you supplied one.)
2. Run the command

```

javadoc -d docDirectory nameOfPackage

```

for a single package. Or, run

```

javadoc -d docDirectory nameOfPackage1 nameOfPackage2. . .

```

to document multiple packages. If your files are in the unnamed package, run instead

```

javadoc -d docDirectory *.java

```

If you omit the `-d docDirectory` option, the HTML files are extracted to the current directory. That can get messy, and I don't recommend it.

The `javadoc` program can be fine-tuned by numerous command-line options. For example, you can use the `-author` and `-version` options to include the `@author` and `@version` tags in the documentation. (By default, they are omitted.) Another useful option is `-link`, to include hyperlinks to standard classes. For example, if you use the command

```
javadoc -link https://docs.oracle.com/en/java/javase/21/docs/api *.java
```

all standard library classes are automatically linked to the documentation on the Oracle web site.

If you use the `-linksource` option, each source file is converted to HTML (without color coding, but with line numbers), and each class and method name turns into a hyperlink to the source.

You can also supply an overview comment for all source files. Place it in a file such as `overview.html` and run the `javadoc` tool with the command line option `-overview filename`. All text between the tags `<body> . . . </body>` is extracted. The content is displayed when the user selects "Overview" from the navigation bar.

For additional options, refer to the online documentation of the `javadoc` utility at <https://docs.oracle.com/en/java/javase/21/javadoc/index.html>.

4.11. Class Design Hints

Without trying to be comprehensive or tedious, I want to end this chapter with some hints that will make your classes more acceptable in well-mannered OOP circles.

1. *Always keep data private.*

This is first and foremost; doing anything else violates encapsulation. You may need to write an accessor or mutator method occasionally, but you are still better off keeping the instance fields private. Bitter experience shows that the data representation may change, but how this data are used will change much less frequently. When data are kept private, changes in their representation will not affect the users of the class, and bugs are easier to detect.

2. *Always initialize data.*

Java won't initialize local variables for you, but it will initialize instance fields of objects. Don't rely on the defaults, but initialize all variables explicitly, either by supplying a default or by setting defaults in all constructors.

3. *Don't use too many basic types in a class.*

The idea is to replace multiple *related* uses of basic types with other classes. This keeps your classes easier to understand and to change. For example, replace the following instance fields in a `Customer` class:

```
private String street;  
private String city;  
private String state;  
private int zip;
```